

Министерство образования и науки Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра комплексной информационной безопасности
электронно-вычислительных систем (КИБЭВС)

Н.А. Новгородова

**SQL ОПЕРАТОРЫ ДЛЯ БД СУБД MICROSOFT SQL SERVER
В ПРИЛОЖЕНИИ MICROSOFT VISUAL STUDIO**

**Учебно-методические указания к выполнению лабораторных работ
по дисциплинам
«Безопасность систем баз данных»,
«Базы данных и экспертные системы»,
«Базы данных»**

Томск 2015

ЛАБОРАТОРНАЯ РАБОТА №4

SQL ОПЕРАТОРЫ ДЛЯ БД СУБД MICROSOFT SQL SERVER В ПРИЛОЖЕНИИ MICROSOFT VISUAL STUDIO

Цель работы:

Изучение основных особенностей формирования запросов в приложении Microsoft Visual Studio для своей предметной области (на примере базы данных для кафе).

1 Краткие сведения об SQL (Structured Query Language)

SQL содержит операторы определения данных, операторы манипулирования данными, операторы, предназначенные для администрирования БД и др.

SQL является международным стандартом, признанным такими организациями по стандартизации, как Международная организация по стандартизации (International Organization for Standardization, **ISO**), Американский национальный институт стандартов (American National Standards Institute, **ANSI**), который входит в **ISO** и Международную электротехническую комиссию (МЭК, International Electrotechnical Commission, **IEC**).

Современные реляционные СУБД содержат в своем составе SQL, соответствующий одному из стандартов. Разработчики СУБД при реализации языка SQL могут вносить в него расширения (дополнительно к базовому набору команд стандарта SQL).

СУБД Microsoft SQL Server в качестве языка запросов использует версию SQL, получившую название Transact-SQL (сокращённо T-SQL). Transact-SQL является реализацией стандарта ANSI/ISO по структурированному языку запросов (SQL) с расширениями и особенностями.

В лабораторной работе рассматриваются SQL операторы манипулирования данными.

Существует четыре основных типа запросов данных в SQL, которые относятся к так называемому **языку манипулирования данными** (Data Manipulation Language или DML):

- **SELECT** – выбрать строки;
- **INSERT** – добавить строки;
- **UPDATE** – изменить строки;
- **DELETE** – удалить строки.

1.1 Оператор выбора SELECT

Для выборки данных используется оператор SELECT. Его синтаксис приведен ниже.

SELECT

[ALL | DISTINCT] [<псевдоним>.]<выражение>[AS<колонка>]
[,<псевдоним>.] <выражение>[AS<колонка>]...]

FROM <откуда берутся данные>

[WHERE <условие/условия отбора>]

[GROUP BY <колонка>[,<колонка>...]]

[HAVING <условие/условия отбора>]

[ORDER BY <колонка>[,<колонка>...]]

Здесь ALL означает, что в результирующую выборку будут включаться все записи, удовлетворяющие запросу. DISTINCT предотвращает повторный вывод одних и тех же записей.

<выражение> может содержать:

- поле базы данных;
- константу; указанное значение константы появится в каждом ряду результата запроса;
- выражение, которое может включать пользовательскую функцию.

Звёздочка «*» указывает, что в результат запроса надо включить все столбцы той или иной таблицы.

Необязательная фраза AS <колонка> специфицирует заголовок колонки в выводе результата запроса. Это полезно, когда имеются <выражения> или содержатся функции работы с полями и требуется присвоить колонке в результате соответствующее имя.

<колонка> может быть выражением.

Если не устраивают имена колонок данных по умолчанию, можно их обозначить по-своему, используя выражение:

[AS <Новое наименование колонки>]

Псевдоним БД, откуда берется нужное поле, может быть назначен в команде SELECT. Вне команды действия не имеет.

Каждой таблице, используемой в запросе, можно присвоить краткое имя (например, просто букву), а затем обращаться по псевдониму:

```
SELECT псевдоним1.колонка1, псевдоним2.колонка2, ....  
FROM таблица1 псевдоним1 [, <таблица2 псевдоним2>...]
```

FROM задается всегда. Определяет имена всех баз данных, таблиц, запросов, которые принимают участие в запросе.

Отбор данных в выборке производится посредством ключевого слова **WHERE**:

WHERE <условие/условия отбора>

<условие отбора> специфицирует критерий, которому должны удовлетворять записи, чтобы попасть в результат действия запроса. Условия фильтрации можно соединять операторами AND, OR. Можно также использовать оператор NOT для реверса значения логического выражения. Также могут быть включены такие операторы, как =, >, <, >=, <=, BETWEEN, LIKE, IN и др.

Примеры операторов:

Оператор	Описание	Пример															
=	Равно	Цена =1000 записи с ценой равной 1000 Тип = 'Фрукты' записи с типом фрукты															
<>	Не равно	Количество <>500 записи с количеством не равным 500															
>	Больше	Цена >1000 записи с ценой больше 1000															
<	Меньше	Количество <10 записи в количестве менее 10															
>=	Больше или равно	Цена >=1000 записи с ценой больше или равной 1000															
<=	Меньше или равно	Количество <=10 записи в количестве менее или равным 10															
AND	Логическое И Таблица истинности: <table><tr><th>A</th><th>B</th><th>A AND B</th></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	A	B	A AND B	1	1	1	1	0	0	0	1	0	0	0	0	Цена>=100 AND Цена <=500 записи с ценой от 100 до 500 включительно
A	B	A AND B															
1	1	1															
1	0	0															
0	1	0															
0	0	0															
OR	Логическое ИЛИ Таблица истинности: <table><tr><th>A</th><th>B</th><th>A OR B</th></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr></table>	A	B	A OR B	1	1	1	1	0	1	0	1	1	0	0	0	Тип = 'Овощи' OR Тип = 'Фрукты' записи с типом фрукты или овощи
A	B	A OR B															
1	1	1															
1	0	1															
0	1	1															
0	0	0															
NOT	Реверс значения логического выражения Таблица истинности: <table><tr><th>A</th><th>NOT A</th></tr><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td></tr></table>	A	NOT A	1	0	0	1	Тип NOT 'Фрукты' типы кроме фруктов									
A	NOT A																
1	0																
0	1																
LIKE	Для отбора данных по шаблону.	Название LIKE ' a%'															

	Синтаксис: <выражение>LIKE<шаблон> При этом в шаблоне используются обозначения: _ - один символ, % - любое количество символов, [список] - любой символ из списка.	записи, в названии которых вторая буква а Фамилия LIKE '[А,Б]%' фамилии, начинающиеся на букву А или Б Тип LIKE '[А-Д]%' записи с типом, начинающимся на буквы от А до Д
IN	Позволяет выбрать значения из заданного списка. Синтаксис: <выражение>IN(<выражение>, ;<выражение>, ;...)	Пол IN ('мужской'; 'женский') записи с полом мужской или женский
BETWEEN	Проверяет, находится ли (или нет) выражение в заданном диапазоне. Синтаксис: <выражение>BETWEEN <нижнее значение> AND <верхнее значение>	Цена BETWEEN 1000 AND 12000 записи с ценой от 1000 до 12000 включите
И др.		

Операторы условия =, >, <, >=, <=, !=, < > применяются к данным имеющим тип: числовой, символьный, дата.

Использование операции сцепления строк

Операция + соединяет два строковых значения, которые могут быть представлены выражениями:

<строковое выражение1> + <строковое выражение2>

Эту операцию можно использовать как после слова SELECT для указания возвращаемых значений, так и в предложении WHERE.

Например:

SELECT Фамилия + ' ' + Имя + ' ' + Отчество AS ФИО
 FROM Сотрудники

Агрегирующие (статистические) функции

Агрегирующая функция рассматривает множество строк таблицы и выдает только одно значение. Приведем список агрегирующих функций.

SUM([DISTINCT] выражение) — сумма (различных) значений в выражении.

AVG([DISTINCT] выражение) — средняя величина (различных) значений в выражении.

COUNT([DISTINCT] выражение) — количество (различных) непустых строк в выражении.

COUNT(*) — полное количество выбранных строк, включая пустые строки и дубликаты.

MAX(выражение) — максимальное значение в выражении.

MIN(выражение) — минимальное значение в выражении.

При использовании агрегирующих функций необходимо учитывать следующее:

- их можно записывать в списке выбора, в предложении HAVING, ORDER BY;
- нельзя применять в предложении WHERE без подзапросов;
- если определено предложение DISTINCT, то повторяющиеся значения аргумента в подсчете значения функции не участвуют;
- если аргумент содержит NULL значения, то функция COUNT всегда возвращает значение 0, а другие агрегирующие функции — NULL значение;
- функции SUM и AVG работают только с аргументами числовых типов;
- функции COUNT, MAX, MIN могут использоваться с аргументами различных типов (числовые, текстовые, дата и др.).

Группировка данных

Для группировки данных используется предложение **GROUP BY**, которое разделяет таблицу на наборы.

GROUP BY неразрывно связано с агрегирующими функциями и предложением HAVING.

Например:

```
SELECT AVG (Цена_меню) AS Средняя_цена  
FROM Блюда
```

```
SELECT Тип, AVG (Цена_меню) AS Средняя_цена  
FROM Блюда  
GROUP BY Тип
```

В результате первого запроса получим среднюю стоимость блюд в меню. Это будет единственное значение. Во втором запросе блюда группируются по типу. Строки помещаются в одну группу тогда и только тогда, когда у них совпадает поле Тип. Затем фраза SELECT применяется к каждой группе, т.е. внутри каждой группы считается средняя цена.

Как правило, в большинстве реализаций SQL элемент из списка GROUP BY должен присутствовать в списке выбора SELECT.

Можно создавать группы внутри других групп.

Если выполнить предложение GROUP BY без агрегирующих функций, то оно будет напоминать предложение DISTINCT, т.е. разделять таблицу на

группы и из группы брать одно значение. Фактически предложение GROUP BY и агрегирующие функции были созданы друг для друга.

GROUP BY возможно применять вместе с предложением WHERE. В этом случае сначала находятся все строки, удовлетворяющие предложению WHERE, затем оставшиеся строки группируются в соответствии с предложением GROUP BY.

Условие, накладываемое на группировки, задается предложением HAVING. Предложение HAVING похоже на предложение WHERE, но HAVING работает не с отдельными записями таблицы, а с группами. Последовательность действий следующая. Сначала для всего запроса выполняется предложение WHERE, затем GROUP BY и уже к сгруппированным данным применяется условие, записанное в предложении HAVING.

Пусть необходимо группировать данные по типу блюд и при этом исключить из рассмотрения наборы, содержащие только одно блюдо:

```
SELECT Тип, COUNT(Код_блюда) AS Количество_блюд
FROM Блюда
GROUP BY Тип
HAVING COUNT(Код_блюда) > 1
```

Условия в предложении HAVING могут объединяться с помощью операторов AND, OR.

Сортировка результатов запроса

Упорядочивание/сортировка результатов запроса производится с использованием ключевого слова **ORDER BY**.

ORDER BY <колонка> [ASC/DESC][,<колонка>,...] — задает упорядочение по колонке, колонкам.

Фраза DESC вызывает упорядочение по убыванию значения соответствующей <колонки>. Фраза ASC специфицирует упорядочение по возрастанию и принимается по умолчанию.

Если указаны два или более столбцов, первый столбец будет использован для глобальной сортировки, второй столбец — для сортировки внутри группы, определяемой единым значением первого столбца, и т.д.

Количество уровней сортировки может быть любым. Порядок сортировки задается в предложении ORDER BY. Порядок следования столбцов при выводе задается списком выбора. В операторе SELECT список выбора может не совпадать со списком сортировки.

Ключевые слова ASC, DESC определяют направление сортировки: по возрастанию, по убыванию соответственно и действует только на один столбец.

При выводе возможна сортировка выражений.

Например:

```
SELECT Тип, COUNT(Код_блюда) AS Количество_блюд
```

```
FROM Блюда
GROUP BY Тип
HAVING COUNT(Код_блюда) > 1
ORDER BY Тип, COUNT(Код_блюда) DESC
```

Использование NULL значений

В том случае, когда информация неполна, неизвестна на настоящий момент времени, приходится иметь дело с так называемыми NULL значениями. При появлении NULL значений не действуют стандартные правила сравнения: одно NULL значение не равно другому NULL значению. NULL значения приводят к появлению трехзначной логики в логических выражениях.

Таблица истинностей для трехзначной логики приведена ниже.

a	b	NOT a	a AND b	a OR b
1	1	0	1	1
1	0	0	0	1
1	–	0	–	1
0	1	1	0	1
0	0	1	0	0
0	–	1	0	–
–	1	–	–	1
–	0	–	0	–
–	–	–	–	–

Здесь NULL значение показано прочерком (–).

При вычислениях строки с NULL значением столбца можно извлекать из таблицы с помощью специального условия IS [NOT] NULL.

Например, чтобы найти все блюда с указанным типом можно использовать запрос:

```
SELECT * FROM Блюда
WHERE Тип IS NOT NULL
```

Объединение/Соединение таблиц

Когда в запрос включаются несколько таблиц, для получения нужных результатов используются объединения. Объединение помогает вернуть из каждой включенной в запрос таблицы только нужные записи, на основании сведений о том, как эти таблицы связаны с другими таблицами в запросе.

Если фраза FROM определяет более одной таблицы или подзапроса, то все эти таблицы соединяются. По умолчанию объединенная таблица представляет собой **перекрестное соединение** (CROSS JOIN), называемое также **декартовым произведением**.

Например:


```
SELECT  Номер_продажи,  Сотрудники.Код_сотрудника,  Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM Продажи, Сотрудники
```

или

```
SELECT  Номер_продажи,  Сотрудники.Код_сотрудника,  Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM Продажи CROSS JOIN Сотрудники
```

В перекрестном объединении каждая строка из одной таблицы объединяется с каждой строкой из другой таблицы, в результате это приводит к тому, что называется полным объединением или декартовым произведением.

Например:

Пусть в таблице **Продажи** будут записи:

Номер_продажи	Дата	Код_сотрудника	Номер_столика
1	01.03.2010...	2	1 у окна
2	02.03.2010...	2	2 у окна
3	02.03.2010...	1	2 у стены
4	03.03.2010...	2	1 у окна

Пусть в таблице **Сотрудники** будут записи:

Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети
1	Петрова	Ирина	Петровна	Официант	женский	замужем	True
2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	False
3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	True
4	Петров	Иван	Васильевич	Менеджер	мужской	женат	True
5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	False
6	Иванов	Егор	Николаевич	Официант	NULL	NULL	NULL

В результате запроса:

```
SELECT Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+' '+Имя+'
'+Отчество AS ФИО
FROM Продажи, Сотрудники
```

Получится:

Номер_продажи	Код_сотрудника	ФИО
1	1	Петрова Ирина Петровна
1	2	Сидоров Петр Сергеевич
1	3	Иванов Сергей Сергеевич
1	4	Петров Иван Васильевич
1	5	Сидорова Мария Ивановна
1	6	Иванов Егор Николаевич
2	1	Петрова Ирина Петровна
2	2	Сидоров Петр Сергеевич
2	3	Иванов Сергей Сергеевич
2	4	Петров Иван Васильевич
2	5	Сидорова Мария Ивановна
2	6	Иванов Егор Николаевич
3	1	Петрова Ирина Петровна
3	2	Сидоров Петр Сергеевич
3	3	Иванов Сергей Сергеевич
3	4	Петров Иван Васильевич
3	5	Сидорова Мария Ивановна
3	6	Иванов Егор Николаевич
4	1	Петрова Ирина Петровна
4	2	Сидоров Петр Сергеевич
4	3	Иванов Сергей Сергеевич
4	4	Петров Иван Васильевич
4	5	Сидорова Мария Ивановна
4	6	Иванов Егор Николаевич

Как видно из таблицы выборки, в ней находятся недостоверные данные. Например, продажу с номером 1 оформил 2 сотрудник, а в таблице выборки продажу номер 1 оформил каждый из сотрудников.

Чтобы этого избежать, необходимо провести объединение таблиц. В одном операторе SELECT может объединяться несколько таблиц. Объединение, как правило, проводится по ключевым столбцам,

Например:

```
SELECT Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM Продажи, Сотрудники
WHERE Продажи.Код_сотрудника = Сотрудники.Код_сотрудника
```

В результате данного запроса получится выборка с достоверными данными:

Номер_продажи	Код_сотрудника	ФИО
1	2	Сидоров Петр Сергеевич
2	2	Сидоров Петр Сергеевич
3	1	Петрова Ирина Петровна
4	2	Сидоров Петр Сергеевич

Рассмотрим этапы обработки данного запроса. Сначала обрабатывается фраза FROM. Однако в этом случае, поскольку в команде заданы две таблицы, система создает декартово произведение строк этих таблиц. Если в предложении FROM указано более двух таблиц, то создается декартово произведение всех таблиц, перечисленных в команде. После создания большой реляционной таблицы система выполняет предложение WHERE. Каждая строка таблицы, созданной предложением FROM, проверяется на выполнение условия WHERE.

При проектировании структуры базы данных в таблицах нужно предусматривать столбцы, по которым в дальнейшем будет производиться объединение таблиц. Чаще всего для этой цели служат первичные и соответствующие им внешние ключи.

Внутреннее объединение (INNER JOIN) - объединяет записи из таблиц, если связующие поля этих таблиц содержат одинаковые значения.

Если же условие не выполняется, то тогда в результирующем набор данных ничего не добавляется.

Синтаксис:

```
FROM таблица1 INNER JOIN таблица2
ON таблица1.поле1оператор_сравнения таблица2.поле2
```

где:

- таблица1, таблица2 - имена таблиц, записи которых объединяются;
- поле1, поле2 - имена связываемых полей. Поля должны относиться к одному типу данных и содержать данные одного вида. Имена этих полей могут быть разными;
- оператор_сравнения - любой оператор сравнения: =, <, >, <=, >=, <>.

Например:

```
SELECT  Номер_продажи,  Сотрудники.Код_сотрудника,  Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM    Сотрудники      INNER      JOIN      Продажи      ON
Сотрудники.Код_сотрудника = Продажи.Код_сотрудника
```

В результате:

Номер_продажи	Код_сотрудника	ФИО
1	2	Сидоров Петр Сергеевич
2	2	Сидоров Петр Сергеевич
3	1	Петрова Ирина Петровна
4	2	Сидоров Петр Сергеевич

В INNER JOIN можно без WHERE:

```

SELECT Сотрудники.Код_сотрудника,  Фамилия,  Имя,  Отчество,
Должность, Номер_продажи
FROM      Сотрудники      INNER      JOIN      Продажи      ON
(Сотрудники.Код_сотрудника=Продажи.Код_сотрудника
Фамилия='Сидоров' AND Должность='Официант')

```

Результат:

Код_сотрудника	Фамилия	Имя	Отчество	Должность	Номер_продажи
2	Сидоров	Петр	Сергеевич	Официант	1
2	Сидоров	Петр	Сергеевич	Официант	2
2	Сидоров	Петр	Сергеевич	Официант	4

Внешние объединения включают записи в результирующую таблицу выборку, даже если условие соединения не выполняется. В этом случае столбцы одной из таблиц возвращаются как обычно, а столбцы из другой таблицы содержат значения NULL. Это удобно для поиска пропущенных значений и нарушенных связей между таблицами.

Внешние объединения являются направленными, они могут быть левыми или правыми.

При **левом объединении (LEFT OUTER JOIN)** – в запрос включаются все строки из первой таблицы (левой) в предложении FROM инструкции SQL и те строки из другой таблицы (правой), в которых значения связанных полей совпадают, если таковые имеются, или NULL – если их нет.

При **правом объединении (RIGHT OUTER JOIN)** – в запрос включаются все строки из второй таблицы (правой) в предложении FROM инструкции SQL и те строки из другой таблицы, в которых значения связанных полей совпадают, если таковые имеются, или NULL – если их нет.

OUTER можно опускать, т.е.: **LEFT JOIN, RIGHT JOIN.**

Синтаксис:

```

FROM таблица1 [LEFT | RIGHT] JOIN таблица2
ON таблица1.поле1оператор_сравнения таблица2.поле2

```

Например:

```

SELECT Номер_продажи,  Сотрудники.Код_сотрудника,  Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM      Сотрудники      LEFT      JOIN      Продажи      ON
Сотрудники.Код_сотрудника = Продажи.Код_сотрудника

```

Результат:

Номер_продажи	Код_сотрудника	ФИО
3	1	Петрова Ирина Петровна
1	2	Сидоров Петр Сергеевич
2	2	Сидоров Петр Сергеевич
4	2	Сидоров Петр Сергеевич
NULL	3	Иванов Сергей Сергеевич
NULL	4	Петров Иван Васильевич
NULL	5	Сидорова Мария Ивановна
NULL	6	Иванов Егор Николаевич

Что соответствует данным БД, так как сотрудники с кодами 3,4,5,6 продажи не оформляли.

```
SELECT Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM Продажи RIGHT JOIN Сотрудники ON Продажи.Код_сотрудника
= Сотрудники.Код_сотрудника
```

Результат:

Номер_продажи	Код_сотрудника	ФИО
3	1	Петрова Ирина Петровна
1	2	Сидоров Петр Сергеевич
2	2	Сидоров Петр Сергеевич
4	2	Сидоров Петр Сергеевич
NULL	3	Иванов Сергей Сергеевич
NULL	4	Петров Иван Васильевич
NULL	5	Сидорова Мария Ивановна
NULL	6	Иванов Егор Николаевич

UNION – объединение результатов выполнения нескольких операторов SELECT

Оператор UNION используется для одновременного просмотра всех данных, возвращаемых несколькими сходными запросами на выборку, в виде объединенного набора.

Оператор UNION позволяет объединить несколько инструкций SELECT в одну. Объединяемые инструкции SELECT должны иметь одинаковое число и порядок выходных полей с такими же или совместимыми типами данных. При выполнении запроса данные из каждого набора соответствующих полей объединяются в одно выходное поле, поэтому выходные данные запроса имеют столько же полей, сколько и каждая инструкция SELECT по отдельности.

Например:

```
SELECT Код_сотрудника, Фамилия, Имя, Отчество, Должность FROM
Сотрудники
WHERE Должность ='Менеджер'
```

```

UNION
SELECT Код_сотрудника, Фамилия, Имя, Отчество, Должность FROM
Сотрудники
WHERE Фамилия ='Сидоров'

```

Результат:

Код_сотрудника	Фамилия	Имя	Отчество	Должность
2	Сидоров	Петр	Сергеевич	Официант
4	Петров	Иван	Васильевич	Менеджер

1.2 Операторы изменения данных

1.2.1 Добавление записей/строк

Для добавления записей/строк используется оператор **INSERT**, который позволяет добавлять строки с помощью ключевого слова **VALUES** или с помощью оператора **SELECT**.

```

INSERT INTO имя_таблицы [(столбец1 [,столбец2] . . .)]
VALUES (константа1 [, константа2] . . . )

```

Применяя данный оператор, необходимо помнить, что вставляемые значения должны иметь тип соответствующий типу столбца, в который значение вставляется или же система должна уметь автоматически производить преобразование. Последовательности столбцов в таблицах должны быть согласованными. Если они не согласованы, операция **INSERT** либо не выполниться, либо выполниться не полностью. И при этом данные могут быть размещены в неверных столбцах.

При использовании оператора **SELECT** в операторе **INSERT** синтаксис несколько изменяется.

```

INSERT INTO имя_таблицы [(вставляемый список столбцов)]
SELECT список столбцов
FROM список таблиц
WHERE условия

```

Оператор **SELECT** позволяет взять данные из нескольких таблиц и вставить их в другую таблицу.

1.2.2 Изменение записей/строк

Изменение существующих данных производится с помощью оператора **UPDATE**:

```
UPDATE имя_таблицы  
SET имя_столбца = выражение [...]  
[WHERE условие]
```

Если добавляемое значение имеет неверный тип или нарушаются другие ограничения на целостность, система обычно запрещает выполнение команды и выдает сообщение об ошибке. В предложении SET определяются столбцы и задаются их новые значения.

Предложение WHERE в операторе UPDATE определяет строки, которые будут изменяться. Предложение WHERE в операторе UPDATE аналогично предложению WHERE в операторе SELECT и так же может включать в себя подзапросы к одной или нескольким таблицам.

1.2.3 Удаление записей/строк

Удаление данных осуществляется с помощью оператора **DELETE**:

```
DELETE FROM имя_таблицы  
WHERE условие
```

В предложении WHERE определяются строки подлежащие удалению. При отсутствии предложения WHERE из таблицы удалятся все строки, если не будет нарушено ограничение ссылочной целостности.

1.3 Подзапросы

Операторы SQL можно вкладывать друг в друга.

Подзапрос (подчиненный запрос) – это оператор SELECT, помещенный в другой оператор SELECT (в список выбора, в предложение WHERE и др.), INSERT, DELETE, UPDATE или в другой подзапрос.

Подзапрос заключен в круглые скобки. Подзапросы имеют две формы:

- **коррелированный подзапрос** (соотнесенный подзапрос) - внутренний запрос для своего выполнения должен получить данные из внешнего запроса. В коррелированном подзапросе внутренний запрос не может быть реализован немедленно: он ссылается на внешний запрос и выполняется поочередно для каждой строки во внешнем запросе;

- **некоррелированный подзапрос** - внутренний запрос выполняется независимо, передавая результаты во внешний запрос.

Например: Найти названия блюд в продаже номер 1:

Некоррелированный подзапрос:

```
SELECT Название
```

```
FROM Блюда
WHERE Код_блюда IN
    (SELECT код_блюда
     FROM Продано
     WHERE Номер_продажи =1)
```



Коррелированный подзапрос:

```
SELECT Название
FROM Блюда
WHERE 1 IN
    (SELECT Номер_продажи
     FROM Продано
     WHERE Код_блюда =Блюда.Код_блюда)
```



Составить перечень блюд с ценами выше средней:

```
SELECT * FROM Блюда
WHERE Цена_меню > (SELECT AVG(Цена_меню) FROM Блюда)
```


2 Особенности формирования запросов в Microsoft Visual Studio

Как уже было сказано ранее (см. лабораторную работу №2), при формировании наборов данных в Microsoft Visual Studio, а также для работы с данными выполняются соответствующие SQL операторы.

Данные SQL операторы необходимо определить в свойствах:

SelectCommand — выбрать данные;

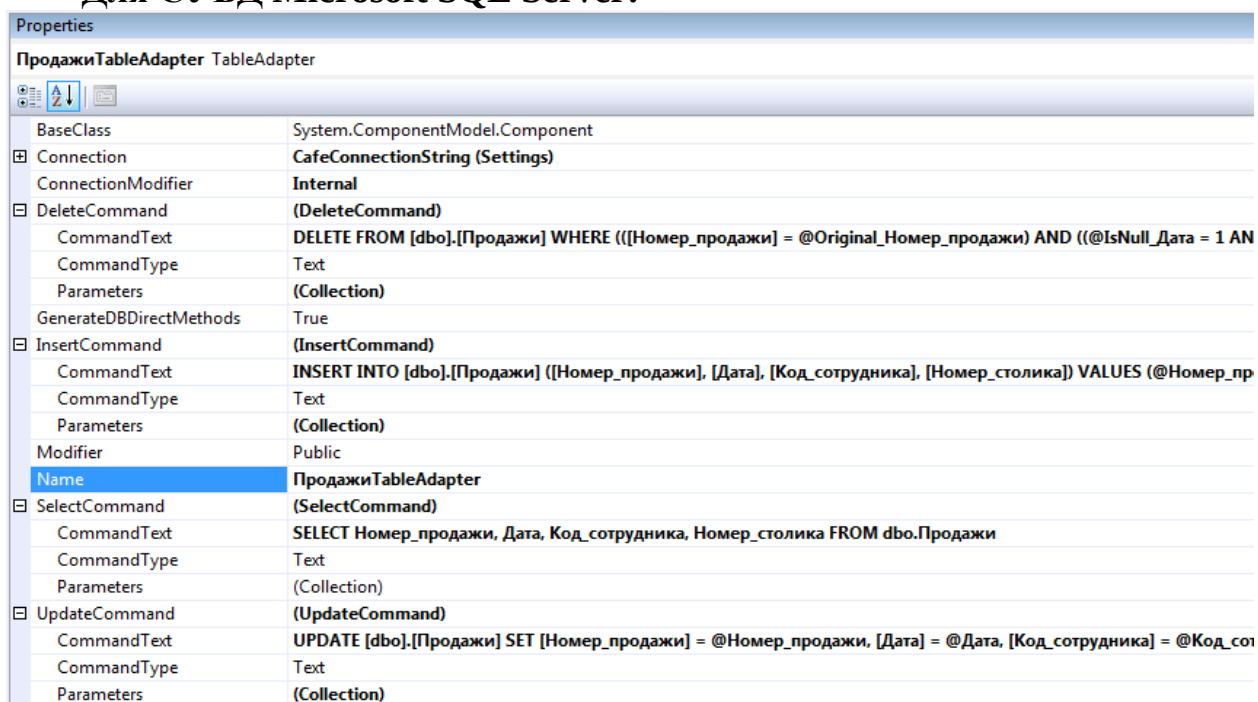
InsertCommand – добавить данные;

DeleteCommand — удалить данные;

UpdateCommand — изменить данные.

Например, для таблицы Продажи:

Для СУБД Microsoft SQL Server:



Properties	
ПродажиTableAdapter TableAdapter	
BaseClass	System.ComponentModel.Component
Connection	CafeConnectionString (Settings)
ConnectionModifier	Internal
DeleteCommand	(DeleteCommand)
CommandText	DELETE FROM [dbo].[Продажи] WHERE (((Номер_продажи) = @Original_Номер_продажи) AND ((@IsNull_Дата = 1 AND
CommandType	Text
Parameters	(Collection)
GenerateDBDirectMethods	True
InsertCommand	(InsertCommand)
CommandText	INSERT INTO [dbo].[Продажи] ([Номер_продажи], [Дата], [Код_сотрудника], [Номер_столика]) VALUES (@Номер_пр
CommandType	Text
Parameters	(Collection)
Modifier	Public
Name	ПродажиTableAdapter
SelectCommand	(SelectCommand)
CommandText	SELECT Номер_продажи, Дата, Код_сотрудника, Номер_столика FROM dbo.Продажи
CommandType	Text
Parameters	(Collection)
UpdateCommand	(UpdateCommand)
CommandText	UPDATE [dbo].[Продажи] SET [Номер_продажи] = @Номер_продажи, [Дата] = @Дата, [Код_сотрудника] = @Код_сот
CommandType	Text
Parameters	(Collection)

При задании SQL запросов необходимо определить в соответствующих свойствах:

- **CommandText** – сам текст SQL оператора;
- **CommandType** – тип;
- **Parameters** – параметры.

Таким образом, в разрабатываемом приложении нами уже используются SQL запросы.

Рассмотрим, как можно еще использовать запросы в приложении.

Статический запрос характерен тем, что описывающий его SQL-оператор не изменяется в процессе выполнения приложения.

Динамическим (параметрическим) является запрос, в SQL-операторе которого в процессе выполнения приложения могут изменяться отдельные его составляющие. В этом случае изменяемая часть оператора оформляется как **Параметры**.

Например, пусть в процессе выполнения приложения может быть выдан запрос: выдать все записи из таблицы Блюда для определенного типа блюд.

Например, для десертов:
SELECT * FROM Блюда
WHERE Тип = 'Десерты'

Для салатов:
SELECT * FROM Блюда
WHERE Тип = 'Салаты'

Также аналогичный запрос по другим типам блюд, которые могут задаваться пользователем. Чтобы каждый раз не писать множество статических запросов, необходимо реализовать параметрический запрос:

SELECT * FROM Блюда
WHERE Тип = @Type

Под параметром понимается имя, предваренное @

В дальнейшем можно задать нужные значения для параметров через свойство **Parameters** и его методы:

- например для СУБД Microsoft SQL Server:

```
SqlConnection connection = new  
SqlConnection(Properties.Settings.Default.CafeConnectionString);  
SqlCommand command = connection.CreateCommand();  
command.CommandText = "SELECT * FROM Блюда WHERE Тип=@Type";  
command.Parameters.AddWithValue("@Type", textBoxType.Text);  
или  
command.Parameters.Add("@Type", SqlDbType.NVarChar).Value =  
textBoxType.Text;
```

В динамических запросах параметры всегда заменяют значения, которые могут изменяться в процессе выполнения. Имена параметров произвольны и могут не совпадать со значениями полей таблицы, которым они обычно ставятся в соответствие.

Формируемые запросы - такие запросы, текст SQL-оператора которых формируется программно в процессе выполнения приложения. Действия по формированию такого запроса состоят в программном занесении в соответствующее свойство текста SQL-запроса, вид которого зависит от текущей ситуации и определяется рядом условий, а также в последующем его выполнении.

3 Практические задания

Работа продолжается в проекте, созданном при выполнении лабораторных работ №2, №3.

3.1 Создании формы для запросов

Создайте для запросов отдельную форму.

Для этого нажмите на проекте правую кнопку мыши и выберете пункт **Add\Windows Forms**.

Задайте для новой формы имя **FormSQL**.

Укажите для формы **FormSQL** свойство **Text** в **Запросы**.

3.2 Создание примеров запросов на выборку

С палитры компонент **Toolbox** разместите на форму **FormSQL** компонент **tabControl** – многостраничный контейнер. Укажите для него:

- свойство **Name** укажите в **tabControlSQL**;
- свойство **Dock** (расположение) в **Fill** (вся область **FormSQL**).

Для первой страницы **tabPage1** компонента **tabControlSQL** укажите:

- свойство **Name** укажите в **tabPagePrimer**;
- свойство **Text** в **Примеры запросов**.

Покажем на данной странице **tabPagePrimer**, пример использования запросов:

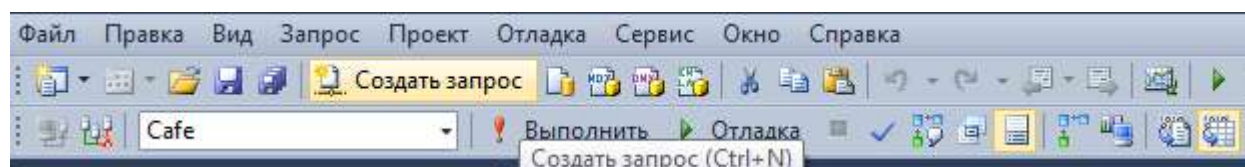
- 1) для **выборки из одной таблицы всех записей и полей** на примере таблицы **Сотрудники**:

```
SELECT * FROM Сотрудники
```

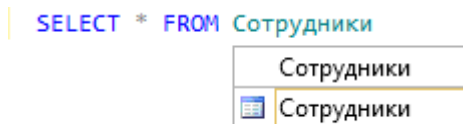
Примечание:

Перед реализацией запросов через интерфейс разрабатываемого приложения для кафе можно протестировать разработанные запросы по необходимым данным, например, в Microsoft SQL Server Management Studio.

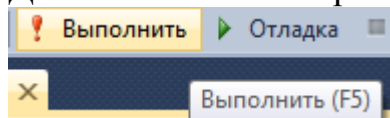
Для этого, например, нажмите на панели инструментов **Создать запрос**:



В появившемся окне напишите необходимый запрос, например:



Для выполнения запроса нажмите, например, **Выполнить**:



И просмотрите результат запроса:

Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети
1	Петрова	Ирина	Петровна	Официант	женский	замужем	True
2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	False
3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	True
4	Петров	Иван	Васильевич	Менеджер	мужской	женат	True
5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	False
6	Иванов	Егор	Николаевич	Официант	NULL	NULL	NULL

2) для **выборки определенных полей из определенной таблицы** на примере таблицы **Блюда**:

Для СУБД Microsoft SQL Server:

```
SELECT Код_блюда, Название+'-'+Тип AS Подробнее, Цена_меню AS [Текущая цена], 'Произойдет' AS Пояснение, Цена_меню*1.5 AS [Повышение цены на 50%]
FROM Блюда
```

Результат:

Код_блюда	Подробнее	Текущая цена	Пояснение	Повышение цены на 50%
1	Грибной-Супы	120,00	Произойдет	180.00000
2	Салат вкусный-Салаты	140,00	Произойдет	210.00000
3	Борщ-Супы	120,00	Произойдет	180.00000
4	Греческий-Салаты	150,00	Произойдет	225.00000
7	Котлеты-Блюда из мяса	130,00	Произойдет	195.00000
8	Торт с вишней-Десерты	150,00	Произойдет	225.00000
9	Коктейль "Весна"-Напитки	200,00	Произойдет	300.00000

Чтобы цена после повышения была представлена с заданной точностью, можно использовать специальные функции приведения типов.

Например, **использование функции CAST**.

Иногда возникает потребность трактовать значение одного типа, как значение другого типа. Например, использовать числовое значение как символьную строку или наоборот. В этом случае применяют функцию

CAST (<значение> AS <тип данных>)

Функция CAST делает копию значения, преобразуя его к указанному типу данных. При этом не следует забывать о множестве типов данных, в которое может быть преобразовано значение.

Например:

```
SELECT Код_блюда, Название+'-'+Тип AS Подробнее, Цена_меню AS
[Текущая цена], 'Произойдет' AS Пояснение, CAST(Цена_меню*1.5 AS
decimal(16,2)) AS [Повышение цены на 50%]
FROM Блюда
```

Примечание:

В Microsoft SQL Server Management Studio:

```
SELECT Код_блюда, Название+'-'+Тип AS Подробнее,
       Цена_меню AS [Текущая цена], 'Произойдет' AS Пояснение,
       CAST(Цена_меню*1.5 AS decimal(16,2)) AS [Повышение цены на 50%]
FROM Блюда
```

Результат:

Код_блюда	Подробнее	Текущая цена	Пояснение	Повышение цены на 50%
1	Грибной-Супы	120,00	Произойдет	180.00
2	Салат вкусный-Салаты	140,00	Произойдет	210.00
3	Борщ-Супы	120,00	Произойдет	180.00
4	Греческий-Салаты	150,00	Произойдет	225.00
7	Котлеты-Блюда из мяса	130,00	Произойдет	195.00
8	Торт с вишней-Десерты	150,00	Произойдет	225.00
9	Коктель "Весна"-Напитки	200,00	Произойдет	300.00

3) для выборки данных по продажам из двух таблиц БД Сотрудники и Продажи. Данные таблицы связаны по Код_сотрудника:

```
SELECT Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM Продажи, Сотрудники
WHERE Продажи.Код_сотрудника = Сотрудники.Код_сотрудника
```

или

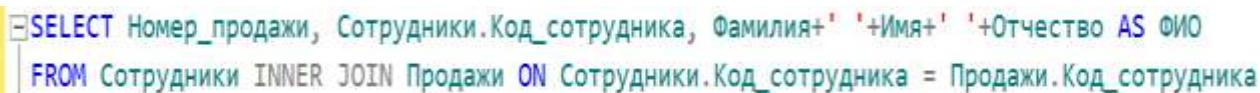
```

SELECT  Номер_продажи,  Сотрудники.Код_сотрудника,  Фамилия+'
'+Имя+' '+Отчество AS ФИО
FROM    Сотрудники      INNER JOIN      Продажи      ON
Сотрудники.Код_сотрудника = Продажи.Код_сотрудника

```

Примечание:

В Microsoft SQL Server Management Studio:



```

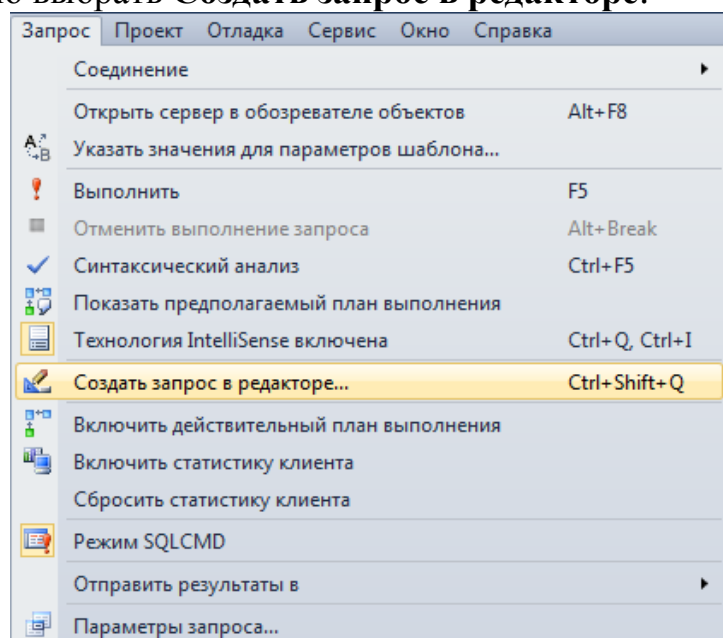
SELECT Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+' '+Имя+' '+Отчество AS ФИО
FROM Сотрудники INNER JOIN Продажи ON Сотрудники.Код_сотрудника = Продажи.Код_сотрудника

```

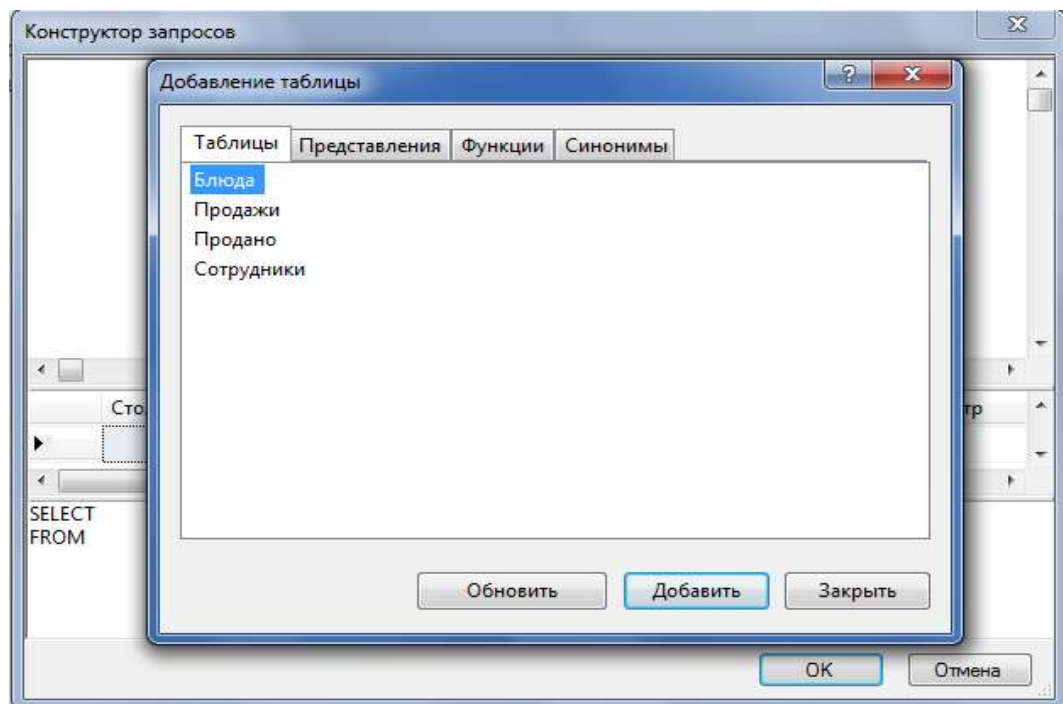
Результат:

Номер_продажи	Код_сотрудника	ФИО
1	2	Сидоров Петр Сергеевич
2	2	Сидоров Петр Сергеевич
3	1	Петрова Ирина Петровна
4	2	Сидоров Петр Сергеевич

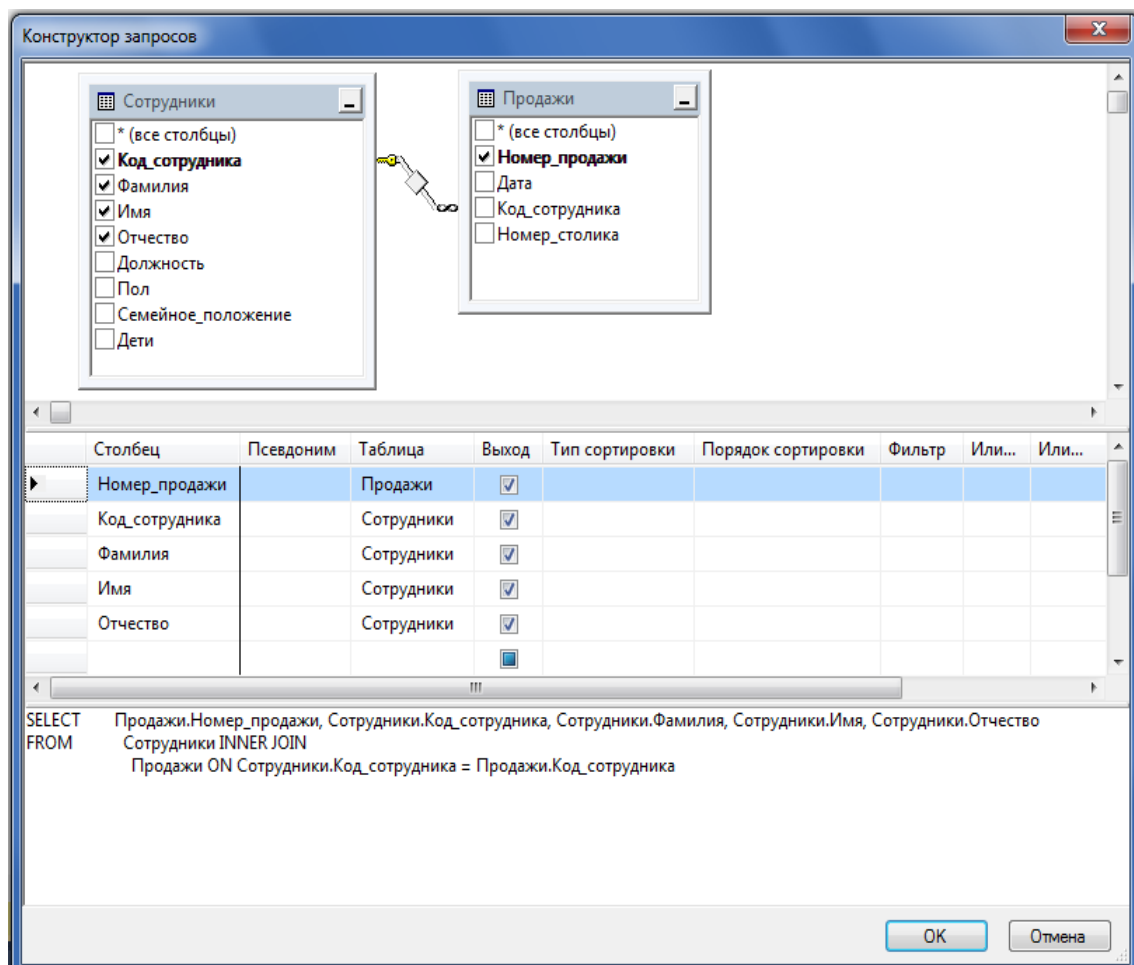
Для запросов можно использовать конструктор запросов. Для этого можно выбрать **Создать запрос в редакторе**:

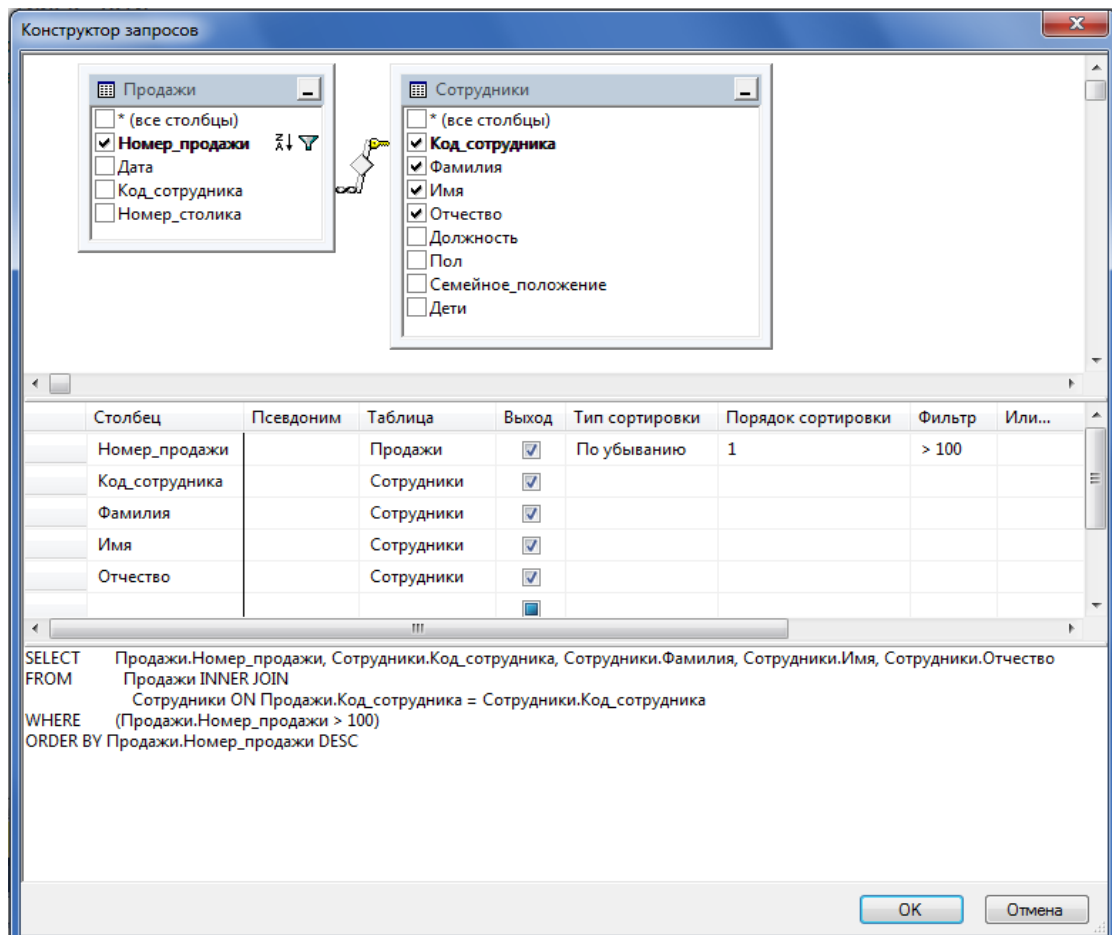


Выбрать необходимые таблицы:



Затем необходимые поля с необходимыми данными:





Реализуем запросы через соответствующий интерфейс программы для кафе.

Разместите на **tabPagePrimer** с палитры компонент **Toolbox** компонент **groupBox**, для которого укажите свойства:

- **Name** в **groupBoxSelect**;
- свойство **Text** в **Запросы по данным**;
- свойство **Dock** (расположение) в **Top**.

Разместите на **groupBoxSelect** с палитры компонент **Toolbox** компонент **radioButton** для первого запроса. Укажите для него свойства:

- **Name** в **radioButtonWorkers**;
- **Text** в **Сотрудники**.

Разместите на **groupBoxSelect** с палитры компонент **Toolbox** компонент **radioButton** для второго запроса. Укажите для него свойства:

- **Name** в **radioButtonDishes**;
- **Text** в **Блюда**.

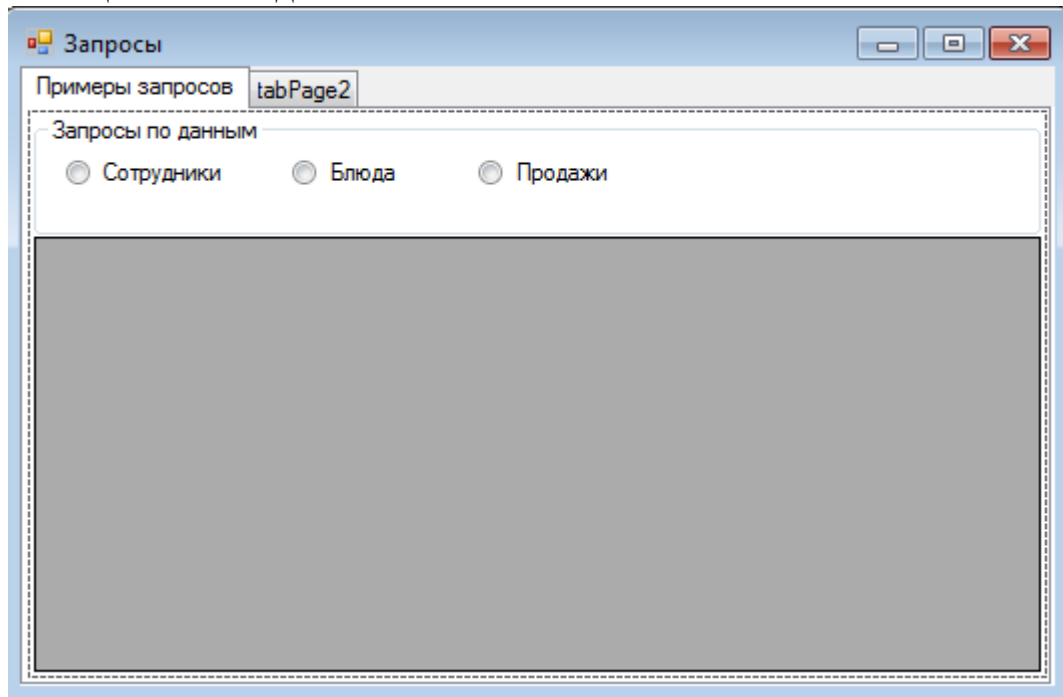
Разместите на **groupBoxSelect** с палитры компонент **Toolbox** компонент **radioButton** для третьего запроса. Укажите для него свойства:

- **Name** в **radioButtonSales**;
- **Text** в **Продажи**.

Под **groupBoxSelect** разместите для отображения данных запросов компонент **dataGridView** с палитры **Toolbox**. Для него укажите свойства:

- **Name** в **dataGridViewSelect**;
- **Dock** в **Fill** (вся область).

Станица имеет вид:



Запрограммируем кнопки Сотрудники, Блюда, Продажи.

Для СУБД Microsoft SQL Server:

Предварительно добавьте пространство имен:

```
using System.Data.SqlClient;
```

```
using System;  
using System.Collections.Generic;  
using System.ComponentModel;  
using System.Data;  
using System.Drawing;  
using System.Text;  
using System.Windows.Forms;  
using System.Data.SqlClient;
```

Для того, чтобы были доступны классы `SqlConnection` для доступа к БД Microsoft SQL Server.

Организуем метод **FillDataGridView** для формирования набора данных по запросам в **dataGridView**. Входным параметром для него является строка с SQL запросом (**sqlSelect**), выходным набор данных:

```
//объявляем метод, на вход подаем строку запроса, а возвращаем объект DataTable
DataTable FillDataGridView(string sqlSelect)
{
    //Создаем объект connection класса SqlConnection для соединения с БД
    //CafeConnectionString – строка описания соединения с источником
    данных

    SqlConnection connection = new
SqlConnection(Properties.Settings.Default.CafeConnectionString);
    //Создаем объект command для SQL команды
    SqlCommand command = connection.CreateCommand();
    //Заносим текст SQL запроса через параметр sqlSelect
    command.CommandText = sqlSelect;
    //Создаем объект adapter класса SqlDataAdapter
    SqlDataAdapter adapter = new SqlDataAdapter();
    //Задаем адаптеру нужную команду, в данном случае команду Select
    adapter.SelectCommand = command;
    //Создаем объект table для последующего отображения результата
запроса

    DataTable table = new DataTable();
    //заполним набор данных результатом запроса
    adapter.Fill(table);
    return table;
}
```

По нажатию соответствующих кнопок пропишите формирование строки запроса и вызов нашей функции **FillDataGridView**.

Для кнопки **Сотрудники (radioButtonWorkers)** пропишите:

```
private void radioButtonWorkers_CheckedChanged(object sender, EventArgs e)
{
    dataGridViewSelect.DataSource = FillDataGridView("SELECT * FROM
Сотрудники");
}
```

Для кнопки **Блюда (radioButtonDishes)** пропишите:

```
private void radioButtonDishes_CheckedChanged(object sender, EventArgs e)
{
    dataGridViewSelect.DataSource = FillDataGridView(@"SELECT Код_блюда,
Название+'-'+Тип AS Подробнее, CAST(Цена_меню AS decimal(16,2)) AS [Текущая
цена], 'Произойдет' AS Пояснение, CAST(Цена_меню*1.5 AS decimal(16,2)) AS
[Повышение цены на 50%] FROM Блюда");
}
```

Для кнопки **Продажи (radioButtonSales)** пропишите:

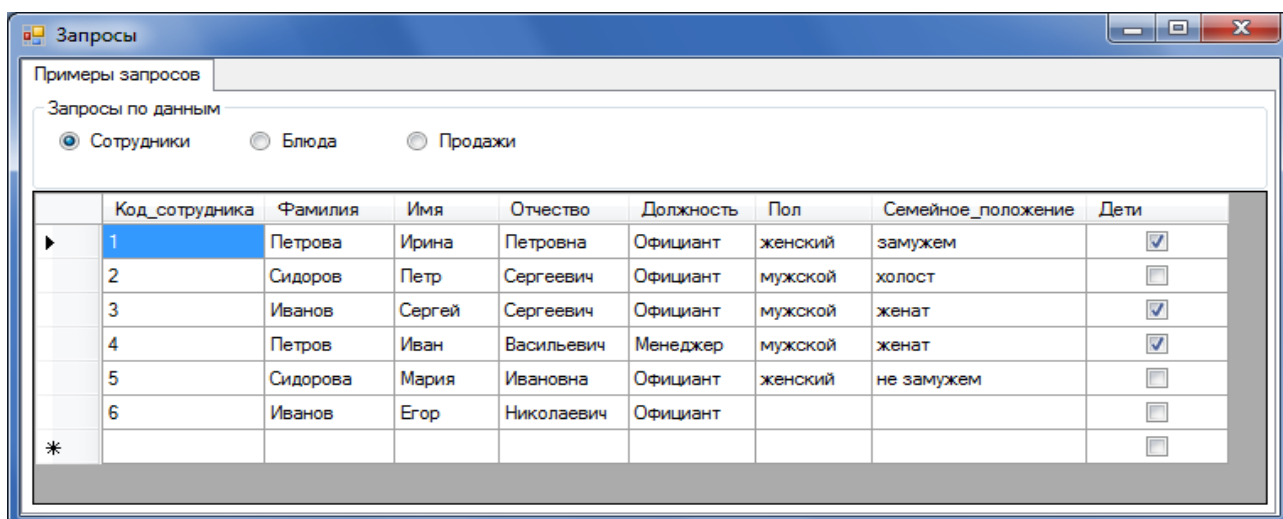
```
private void radioButtonSales_CheckedChanged(object sender, EventArgs e)
{
    dataGridViewSelect.DataSource = FillDataGridView(@"SELECT
Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+' '+Имя+' '+Отчество AS ФИО
FROM Продажи, Сотрудники WHERE Продажи.Код_сотрудника =
Сотрудники.Код_сотрудника");
}
```

ИЛИ

```
private void radioButtonSales_CheckedChanged(object sender, EventArgs e)
{
    dataGridViewSelect.DataSource = FillDataGridView(@"SELECT
Номер_продажи, Сотрудники.Код_сотрудника, Фамилия+' '+Имя+' '+Отчество AS ФИО
FROM Сотрудники INNER JOIN Продажи ON Сотрудники.Код_сотрудника =
Продажи.Код_сотрудника");
}
```

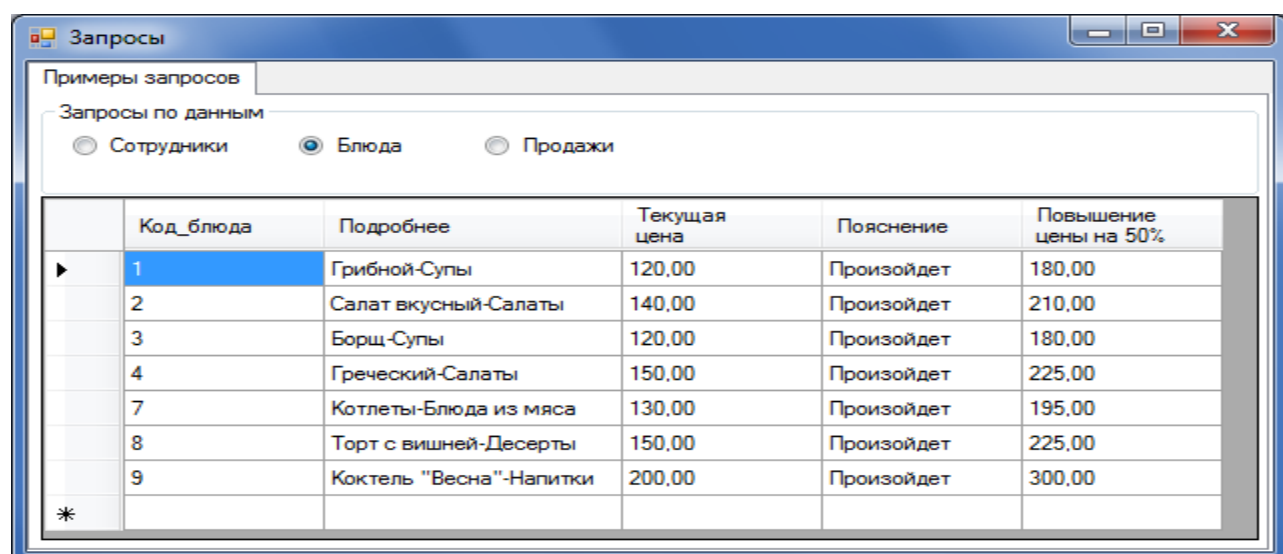
Запустите программу. Посмотрите, как отрабатывают полученные запросы.

Например:



The screenshot shows a window titled 'Запросы' (Queries) with a tab 'Примеры запросов' (Query Examples). Under 'Запросы по данным' (Queries by data), the 'Продажи' (Sales) radio button is selected. The table displays employee data with columns: Код_сотрудника, Фамилия, Имя, Отчество, Должность, Пол, Семейное_положение, and Дети. The first row is highlighted.

	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_положение	Дети
▶	1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒
	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐
	3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒
	4	Петров	Иван	Васильевич	Менеджер	мужской	женат	☒
	5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐
	6	Иванов	Егор	Николаевич	Официант			☐
*								☐



The screenshot shows the same 'Запросы' (Queries) window, but the 'Блюда' (Dishes) radio button is selected. The table displays dish data with columns: Код_блюда, Подробнее, Текущая цена, Пояснение, and Повышение цены на 50%. The first row is highlighted.

	Код_блюда	Подробнее	Текущая цена	Пояснение	Повышение цены на 50%
▶	1	Грибной-Супы	120,00	Произойдет	180,00
	2	Салат вкусный-Салаты	140,00	Произойдет	210,00
	3	Борщ-Супы	120,00	Произойдет	180,00
	4	Греческий-Салаты	150,00	Произойдет	225,00
	7	Котлеты-Блюда из мяса	130,00	Произойдет	195,00
	8	Торт с вишней-Десерты	150,00	Произойдет	225,00
	9	Коктейль "Весна"-Напитки	200,00	Произойдет	300,00
*					

3.3 Полная запись оператора SELECT

Перейдите на страницу **tabPage2**.

Для нее укажите:

- свойство **Name** в **tabPageSelect**;
- свойство **Text** в **Полная запись SELECT**.

На **tabPageSelect** с палитры **Toolbox** разместите компонент **groupBox** для которого укажите свойства:

- **Name** в **groupBoxFSelect**;
- **Text** в **Прибыль сотрудников**;
- **Dock** в **Top**.

Будем просматривать прибыль сотрудника, чья фамилия вводится в строке ввода.

Для этого разместите на **groupBoxFSelect** компоненты с палитры **Toolbox**:

- компонент **label** для которого укажите свойства:
 - **Name** в **labelWorker**;
 - **Text** в **Фамилия сотрудника**;
- рядом с **labelWorker** компонент **textBox**, для которого укажите свойство **Name** в **textBoxWorker**.

Для детализации прибыли сотрудников по продажам и типам разместите на **groupBoxFSelect** компонент **groupBox**, для которого укажите свойства:

- **Name** в **groupBoxDet**;
- **Text** в **Детализация прибыли сотрудников**.

На **groupBoxDet** разместите:

- компонент **radioButton** для детализации по продажам. Укажите для нее свойства:

- **Name** в **radioButtonDet_Sales**;
- **Text** в **Прибыль сотрудника по продажам**.

- компонент **radioButton** для детализации по типам. Укажите для нее свойства:

- **Name** в **radioButtonDet_Type**;
- **Text** в **Прибыль сотрудника по типам**.

- компонент **radioButton** для отсутствия детализации. Укажите для нее свойства:

- **Name** в **radioButtonDet_NO**;
- **Text** в **Нет детализации**.

Будем отбирать данные по прибыли более определенной суммы, вводимой пользователем с клавиатуры.

Для этого разместите на **groupBoxFSelect** компонент **checkBox**, для которого укажите свойства:

- **Name** в **checkBoxMore**;
- **Text** в **Выбрать прибыль более**.

Рядом с **checkBoxMore** разместите строку ввода для ввода необходимой суммы. Для этого с палитры компонент **Toolbox** разместите компонент **textBox**, для которого укажите свойство **Name** в **textBoxMore**.

Включим сортировку по убыванию прибыли. Для этого разместите на **groupBoxFSelect** компонент **checkBox**, для которого укажите свойства:

- **Name** в **checkBoxOrder**;
- **Text** в **Включить сортировку по убыванию прибыли**.

Выполнять запрос будем по нажатию кнопки. Для этого разместите на **groupBoxFSelect** с палитры компонент **Toolbox** компонент **button**, для которого укажите свойства:

- **Name** в **buttonF_select**;
- **Text** в **Прибыль сотрудников**;
- **Image** в соответствующую картинку.

Результат будем представлять в dataGridView. Разместите на **groupBoxFSelect** компонент **dataGridView**. Для него укажите свойства:

- **Dock** в **Fill**;
- **Name** в **dataGridViewFSelect**.

Вид страницы:

Запрограммируем кнопку **Прибыль сотрудников (buttonF_select)**.

Для СУБД Microsoft SQL Server:

```
private void buttonF_select_Click(object sender, EventArgs e)
{
    if (String.IsNullOrEmpty(textBoxWorker.Text))
    {
        MessageBox.Show("Обязательно укажите фамилию необходимого
сотрудника.\nДопустим ввод первых символов.", "Внимание", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        return;
    }
    if (checkBoxMore.Checked && String.IsNullOrEmpty(textBoxMore.Text))
    {
        MessageBox.Show("Не указана прибыль в условии", "Внимание",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        checkBoxMore.Checked = false;
        return;
    }
    string sqlSelect = "";
    if (radioButtonDet_Sales.Checked)
        sqlSelect = @"SELECT w.Код_сотрудника, Фамилия, Имя, Отчество,
s.Номер_продажи,
CAST(Sum(Количество*Цена*(1-Скидка)) AS decimal(16,2)) AS
Прибыль
FROM Сотрудники w, Продажи s, Продано ds, Блюда d
WHERE Фамилия LIKE @last_name AND
w.Код_сотрудника = s.Код_сотрудника AND
s.Номер_продажи = ds.Номер_продажи AND
ds.Код_блюда = d.Код_блюда
GROUP BY w.Код_сотрудника, Фамилия, Имя, Отчество,
s.Номер_продажи";
    else
        if (radioButtonDet_Type.Checked)
            sqlSelect = @"SELECT Сотрудники.Код_сотрудника, Фамилия,
Имя, Отчество, Тип,
CAST(Sum(Количество*Цена*(1-Скидка)) AS decimal(16,2)) AS
Прибыль
FROM Сотрудники INNER JOIN (Продажи INNER JOIN (Продано
INNER JOIN Блюда ON Продано.Код_блюда = Блюда.Код_блюда) ON
Продажи.Номер_продажи = Продано.Номер_продажи)
ON Сотрудники.Код_сотрудника = Продажи.Код_сотрудника
WHERE Фамилия LIKE @last_name
GROUP BY Сотрудники.Код_сотрудника, Фамилия, Имя, Отчество,
Тип";
        else
            sqlSelect = @"SELECT Сотрудники.Код_сотрудника, Фамилия,
Имя, Отчество,
CAST(Sum(Количество*Цена*(1-Скидка)) AS decimal(16,2)) AS
Прибыль
FROM Сотрудники INNER JOIN Продажи ON
Сотрудники.Код_сотрудника=Продажи.Код_сотрудника
INNER JOIN Продано ON
Продажи.Номер_продажи=Продано.Номер_продажи
INNER JOIN Блюда ON Продано.Код_блюда=Блюда.Код_блюда AND
Фамилия LIKE @last_name
GROUP BY Сотрудники.Код_сотрудника, Фамилия, Имя, Отчество";

    if (checkBoxMore.Checked)
        sqlSelect += " HAVING Sum(Количество*Цена*(1-Скидка)) >@amount";
    if (checkBoxOrder.Checked)
```

```

sqlSelect += " ORDER BY Sum(Количество*Цена*(1-Скидка)) desc";

SqlConnection connection = new
SqlConnection(Properties.Settings.Default.CafeConnectionString);
SqlCommand command = connection.CreateCommand();
command.CommandText = sqlSelect;
command.Parameters.AddWithValue("@last_name", textBoxWorker.Text +
"%");

if (checkBoxMore.Checked)
    try
    {
        command.Parameters.Add("@amount", SqlDbType.Money).Value =
Double.Parse(textBoxMore.Text);
    }
    catch
    {
        MessageBox.Show("Прибыль в условии должна быть задана
числом", "ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
        checkBoxMore.Checked = false;
        return;
    }

SqlDataAdapter adapter = new SqlDataAdapter();
adapter.SelectCommand = command;
DataTable table = new DataTable();
adapter.Fill(table);
dataGridViewFSelect.DataSource = table;
if (table.Rows.Count == 0) MessageBox.Show("Нет значений!",
"Информация", MessageBoxButtons.OK, MessageBoxIcon.Information);
}

```

Запустите программу. Посмотрите, как отработывают сформированные запросы.

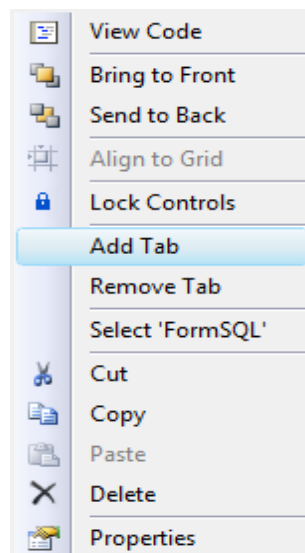
Например:

	Код_сотрудника	Фамилия	Имя	Отчество	Тип	Прибыль
▶	2	Сидоров	Петр	Сергеевич	Супы	1620,00
	2	Сидоров	Петр	Сергеевич	Салаты	710,00
	2	Сидоров	Петр	Сергеевич	Блюда из мяса	390,00
	2	Сидоров	Петр	Сергеевич	Десерты	337,50
*						

3.4 Подзапросы

Покажем пример использования подзапросов.

Добавьте новую станицу на **tabControlSQL**. Для это нажмите на **tabControlSQL** правую кнопку мыши и выберете **Add Tab**:



Задайте для новой страницы:

- свойство **Name** для новой страницы в **tabPageSubquery**;
- свойство **Text** в **Примеры подзапросов**.

Разместите на **tabPageSubquery** с палитры компонент **Toolbox** компонент **groupBox**, для которого укажите свойства:

- **Name** в **groupBoxSubquery**;
- свойство **Text** в **Подзапросы по данным**;
- свойство **Dock** (расположение) в **Top**.

Разместите на **groupBoxSubquery** с палитры компонент **Toolbox** компонент **radioButton** для первого подзапроса. Укажите для него свойства:

- **Name** в **radioButtonCorrelated**;
- **Text** в **Коррелированный подзапрос**.

Разместите на **groupBoxSubquery** с палитры компонент **Toolbox** компонент **radioButton** для второго запроса. Укажите для него свойства:

- **Name** в **radioButtonNoCorrelated**;
- **Text** в **Некоррелированный подзапрос**.

Будем просматривать подзапросы для продажи, чей номер вводится в строке ввода.

Для этого разместите на **groupBoxSubquery** компоненты с палитры **Toolbox**:

- компонент **label** для которого укажите свойства:
 - **Name** в **labelNumber**;
 - **Text** в **Номер продажи**;
- рядом с **labelNumber** компонент **textBox** для указания номера. Для него укажите свойство **Name** в **textBoxNumber**.

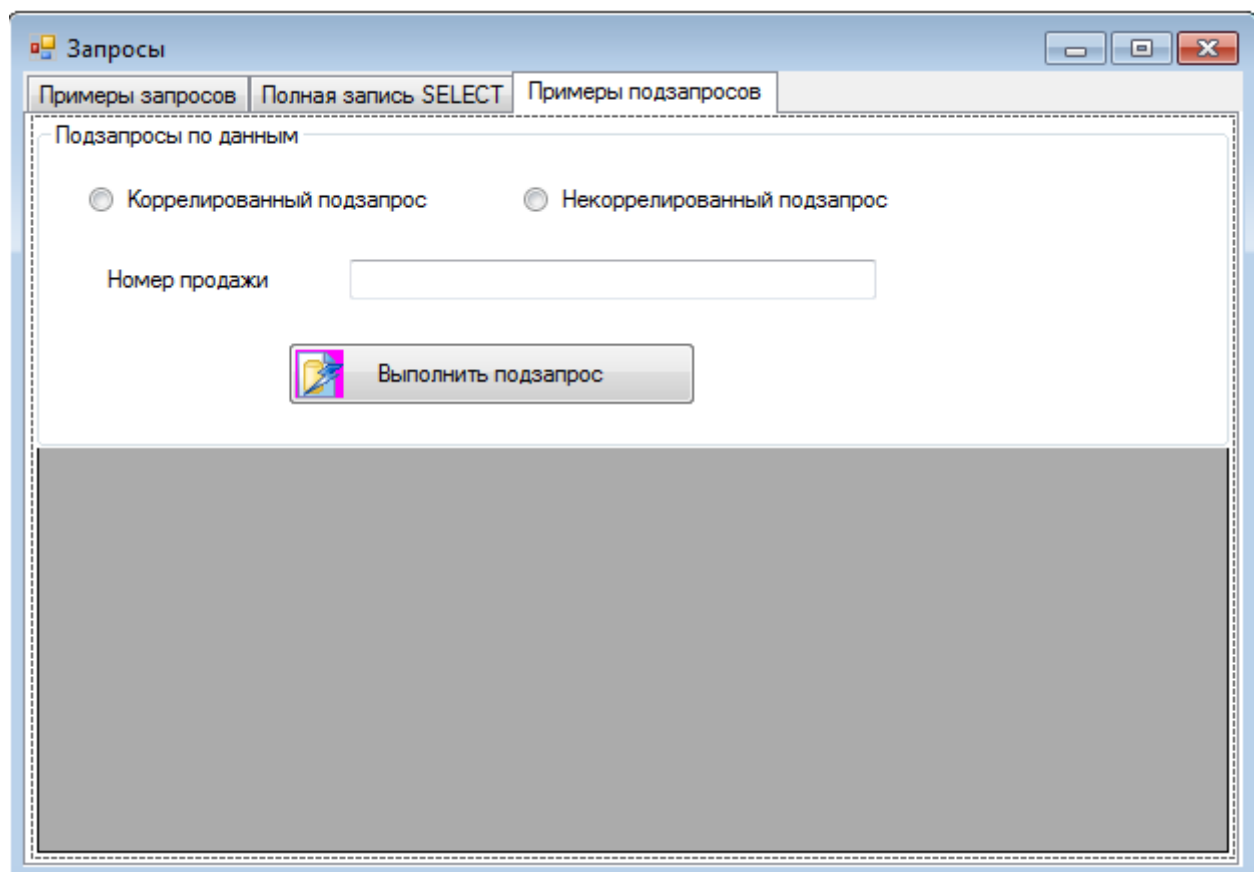
Разместите на **groupBoxSubquery** с палитры компонент **Toolbox** компонент **button** для запуска подзапросов. Укажите для него свойства:

- **Name** в **buttonSubquery**;
- **Text** в **Выполнить подзапрос**;
- **Image** в соответствующую картинку.

Под **groupBoxSubquery** разместите для отображения данных запросов компонент **dataGridView** с палитры **Toolbox**. Для него укажите свойства:

- **Name** в **dataGridViewSubquery**;
- **Dock** в **Fill** (вся область).

Вид страницы:



Запрограммируем кнопку **Выполнить подзапрос** (**buttonSubquery**).

Для СУБД Microsoft SQL Server:

```
private void buttonSubquery_Click(object sender, EventArgs e)
```

```

{
    if (String.IsNullOrEmpty(textBoxNumber.Text))
    {
        MessageBox.Show("Обязательно укажите номер необходимой продажи",
"Внимание", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    string sqlSelect = "";

    if (radioButtonCorrelated.Checked)

        sqlSelect = @"SELECT Продано.Номер_продажи, Продано.Код_блюда,
        (SELECT Название+' '. Тип - '+Тип FROM Блюда WHERE Код_блюда
=Продано.Код_блюда) AS Блюдо,
        CAST(Количество*Цена*(1-Скидка) AS decimal(16,2)) AS Стоимость,
        (SELECT Фамилия+' '+Имя+' '+Отчество FROM Сотрудники
        WHERE Код_сотрудника IN (SELECT Код_сотрудника FROM Продажи
        WHERE Номер_продажи=Продано.Номер_продажи)) AS
Оформил_сотрудник
        FROM Продано
        WHERE Номер_продажи = @number";

    else
        if (radioButtonNoCorrelated.Checked)
            sqlSelect = @"SELECT * FROM Продано
            WHERE Номер_продажи =@number AND Цена > (SELECT AVG(Цена)
FROM Продано)";
        else
        {
            MessageBox.Show("Не выбрали вид подзапроса", "Ошибка",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        SqlConnection connection = new
SqlConnection(Properties.Settings.Default.CafeConnectionString);
        SqlCommand command = connection.CreateCommand();
        command.CommandText = sqlSelect;
        try
        {
            command.Parameters.Add("@number", SqlDbType.Int).Value =
int.Parse(textBoxNumber.Text);
        }
        catch
        {
            MessageBox.Show("Номер продажи в условии должен быть задан
числом", "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        SqlDataAdapter adapter = new SqlDataAdapter();
        adapter.SelectCommand = command;
        DataTable table = new DataTable();
        adapter.Fill(table);
        dataGridViewSubquery.DataSource = table;
        if (table.Rows.Count == 0) MessageBox.Show("Нет значений!",
"Информация", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}

```

Запустите программу. Посмотрите, как обрабатывают сформированные запросы.

Например:

Запросы

Примеры запросов | Полная запись SELECT | Примеры подзапросов

Подзапросы по данным

☒ Коррелированный подзапрос ☐ Некоррелированный подзапрос

Номер продажи:

 Выполнить подзапрос

	Номер_продажи	Код_блюда	Блюдо	Стоимость	Оформил_сотрудник
▶	1	1	Грибной. Тип - Супы	960,00	Сидоров Петр Сергеевич
	1	2	Салат вкусный. Тип - Салаты	560,00	Сидоров Петр Сергеевич
	1	3	Борщ. Тип - Супы	540,00	Сидоров Петр Сергеевич
	1	7	Котлеты. Тип - Блюда из мяса	260,00	Сидоров Петр Сергеевич
	1	8	Торт с вишней. Тип - Десерты	337,50	Сидоров Петр Сергеевич
*					

Запросы

Примеры запросов | Полная запись SELECT | Примеры подзапросов

Подзапросы по данным

☐ Коррелированный подзапрос ☒ Некоррелированный подзапрос

Номер продажи:

 Выполнить подзапрос

	Номер_продажи	Код_блюда	Количество	Цена	Скидка
▶	1	2	4	140,0000	0,00
	1	8	3	150,0000	0,25
*					

3.5 Запросы изменения данных

Покажем пример использования запросов изменения данных (**INSERT**, **UPDATE**, **DELETE**) на примере таблицы **Блюда**.

Добавьте новую станицу на **tabControlSQL**. Для это нажмите на **tabControlSQL** правую кнопку мыши и выберете **Add Tab**.

Задайте для новой страницы:

- свойство **Name** в **tabPageDML**;
- свойство **Text** в **Запросы изменения данных**.

Перейдите на страницу **tabPageDML**.

Разместите с палитры компонент **Toolbox** компонент **groupBox**, для которого укажите свойства:

- **Name** в **groupBoxDML**;
- **Text** в **Операторы**;
- **Dock** в **Top**.

На **groupBoxDML** разместите с палитры компонент **Toolbox**:

- компонент **radioButton** для добавления. Укажите для нее свойства:
 - **Name** в **radioButtonInsert_dish**;
 - **Text** в **Добавить данные по блюду**;
- компонент **radioButton** для редактирования. Укажите для нее свойства:
 - **Name** в **radioButtonUpdate_dish**;
 - **Text** в **Изменить данные по блюду с заданным кодом**.
- компонент **radioButton** для удаления. Укажите для нее свойства:
 - **Name** в **radioButtonDelete_dish**;
 - **Text** в **Удалить данные по блюду с заданным кодом**.

Под **groupBoxDML** разместите с палитры **Toolbox** компонент **label**, для которого укажите свойства:

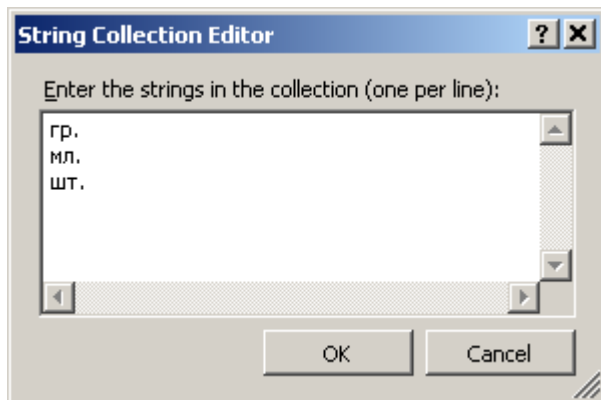
- **Name** в **labelId_dish**;
- **Text** в **Код блюда**;
- рядом с **labelId_dish** компонент **textBox**, для которого укажите свойство **Name** в **textBoxId_dish**.

Так как логика работы добавления, редактирования и удаления будет разной, то для удобства разместим дальнейшие компоненты на компонент панель. Для этого разместите с палитры **Toolbox** компонент **panel**, для которого укажите свойство **Name** в **panelDish**.

На панель **panelDish** разместите с палитры **Toolbox**:

- компонент **label**, для которого укажите свойства:
 - **Name** в **labelName_dish**;

- **Text** в **Название блюда**;
- рядом с **labelName_dish** компонент **textBox**, для которого укажите свойство **Name** в **textBoxName_dish**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelType_dish**;
 - **Text** в **Тип блюда**;
- рядом с **labelType_dish** компонент **textBox**, для которого укажите свойство **Name** в **textBoxType_dish**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelPrice_dish**;
 - **Text** в **Цена блюда**;
- рядом с **labelPrice_dish** компонент **textBox**, для которого укажите свойство **Name** в **textBoxPrice_dish**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelWeight_dish**;
 - **Text** в **Выход(вес) блюда**;
- рядом с **labelWeight_dish** компонент **textBox**, для которого укажите свойство **Name** в **textBoxWeight_dish**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelEd_dish**;
 - **Text** в **Единицы измерения**;
- рядом с **labelEd_dish** компонент **listBox**, для которого укажите свойства:
 - **Name** в **listBoxEd_dish**;
 - **Items** в:



- компонент **checkBox** для которого укажите свойства:
 - **Name** в **checkBoxFirm**;

- **Text** в **Фирменное блюдо**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelPhoto_dish**;
 - **Text** в **Фото блюда**;
- рядом с **labelPhoto_dish** компонент **pictureBox**, для которого укажите свойства:
 - **Name** в **pictureBoxPhoto_dish**;
 - **SizeMode** в **StretchImage**;
- компонент **label** для которого укажите свойства:
 - **Name** в **labelIng_dish**;
 - **Text** в **Состав блюда**;
- рядом с **labelIng_dish** компонент **textBox**, для которого укажите свойства:
 - **Name** в **textBoxIng_dish**;
 - **Multiline** в **True**;
 - **ScrollBars** в **Vertical** или в **Both**.

Для удобства загрузки фотографии блюда из файла разместите рядом с **pictureBoxPhoto_dish** компонент **button**, для которого укажите свойства:

- **Name** в **buttonOpenPhoto_dish**;
- **Text** в **Открыть фото блюда**.

Для загрузки фото из файла положите на форму диалог **openFileDialog**. Укажите для него свойство **Name** в **openFileDialogDish**.

Запрограммируем нажатие кнопки **buttonOpenPhoto_dish** для загрузки фото в **pictureBoxPhoto_dish**:

```
string fileImage = "";

private void buttonOpenPhoto_dish_Click(object sender, EventArgs e)
{
    openFileDialogDish.Title = "Укажите файл для фото";
    if (openFileDialogDish.ShowDialog() == DialogResult.OK)
    {
        fileImage = openFileDialogDish.FileName;
        try
        {
            pictureBoxPhoto_dish.Load(openFileDialogDish.FileName);
        }
        catch
        {
            MessageBox.Show("Выбран не тот формат файла", "Ошибка",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
}
```

```

    }
    else fileImage = "";
}

```

Вызывать выполнение соответствующего запроса будем по нажатию кнопки **Выполнить запрос**.

Для этого разместите с палитры компонент **Toolbox** компонент **button**, для которого укажите свойства:

- **Name** в **buttonExecuteDML**;
- **Text** в **Выполнить запрос**;
- **Image** в соответствующую картинку.

Чтобы отобразить результаты выполнения запросов разместите на **tabPageDML** с палитры **ToolBox** компонент **dataGridView**, для которого свойство **Name** укажите в **dataGridViewDish**.

Для добавления данных в **dataGridViewDish** разместите кнопку **button** с палитры **ToolBox** для которой укажите свойства:

- **Name** в **buttonSelectDishes**;
- **Text** в **Показать список блюд**.

Таким образом, вид страницы имеет вид:

При добавлении данных в таблицу **Блюда** в соответствующие поля будут добавлены соответствующие значения из соответствующих компонент на форме.

Данные значения для ввода могут быть различны, следовательно, запрос будет не с заранее определенными значениями, а с параметрами, которые будут подставляться из соответствующих компонент на форме.

Запрос на добавление будет иметь вид:

```
INSERT INTO Блюда (Код_блюда, Название, Тип, Цена_меню,  
Выход, Единицы, Фирменное_блюдо, Фото, Состав)  
VALUES (@id, @Name, @Type, @Price, @Weight, @Ed, @Firm,  
@Photo, @Sostav)"
```

Где параметры обозначены @.

Оформим данный запрос через соответствующий метод.

Колонки **Код_блюда, Название, Тип, Цена_меню** в данной БД обязательны для заполнения. Если их пользователь не укажет при добавлении нового блюда, то СУБД сгенерирует соответствующее исключение. Чтобы вместо этого сообщения выдать более понятное пользователю сообщение, необходимо отследить данную ситуацию в приложении, когда пользователь для нового блюда не указал Код_блюда, Название, Тип, Цену меню.

Следует отметить, что пользователь может по ошибке задать для колонок **Код_блюда, Цена** и **Вес** не числовые значения. В этом случае СУБД сгенерирует исключение. Чтобы вместо этого сообщения выдать понятное пользователю сообщение, необходимо отследить данную ситуацию в приложении.

Таким образом, получаем следующий метод для добавления данных о блюде **InsertDish()**:

Для СУБД Microsoft SQL Server:

```
void InsertDish()  
{  
    if (String.IsNullOrEmpty(textBoxId_dish.Text) ||  
        (String.IsNullOrEmpty(textBoxName_dish.Text) ||  
         (String.IsNullOrEmpty(textBoxType_dish.Text) ||  
          (String.IsNullOrEmpty(textBoxPrice_dish.Text))))  
    {  
        MessageBox.Show("Обязательно введите код блюда, название, тип и  
цену блюда", "Внимание", MessageBoxButtons.OK, MessageBoxIcon.Warning);  
        return;  
    }  
  
    int id;  
    if (!int.TryParse(textBoxId_dish.Text, out id))
```



```

    {
        MessageBox.Show("Некоректное значение кода блюда!", "Внимание",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    double price = 0;
    if (!double.TryParse(textBoxPrice_dish.Text, out price))
    {
        MessageBox.Show("Некоректное значение цены!", "Внимание",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    double weight = 0;
    if ((!String.IsNullOrEmpty(textBoxWeight_dish.Text)) &&
    (!double.TryParse(textBoxWeight_dish.Text, out weight)))
    {
        MessageBox.Show("Некоректное значение веса!", "Внимание",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    string sqlInsert = @"INSERT INTO Блюда (Код_блюда, Название, Тип,
    Цена_меню, Выход, Единицы, [Фирменное_блюдо], Фото, Состав)
    VALUES (@id, @Name, @Type, @Price, @Weight, @Ed,
    @Firm, @Photo, @Sostav)";
    SqlConnection connection = new
    SqlConnection(Properties.Settings.Default.CafeConnectionString);
    connection.Open();
    SqlCommand command = connection.CreateCommand();
    command.CommandText = sqlInsert;
    command.Parameters.AddWithValue("@id", id);
    command.Parameters.AddWithValue("@Name", textBoxName_dish.Text);
    //или другим способом, если необходимо явное указание типа данных
    command.Parameters.Add("@Type", SqlDbType.NVarChar).Value =
    textBoxType_dish.Text;
    command.Parameters.AddWithValue("@Price", price);
    command.Parameters.AddWithValue("@Weight", weight);

    if (!String.IsNullOrEmpty(listBoxEd_dish.Text))
        command.Parameters.AddWithValue("@Ed", listBoxEd_dish.Text);
    else
        command.Parameters.AddWithValue("@Ed", DBNull.Value);

    command.Parameters.AddWithValue("@Firm", checkBoxFirm.Checked);

    if (!String.IsNullOrEmpty(fileImage))
        command.Parameters.AddWithValue("@Photo",
    File.ReadAllBytes(fileImage));
    else
    {
        command.Parameters.Add("@Photo", SqlDbType.VarBinary);
        command.Parameters["@Photo"].Value = DBNull.Value;
    }

    command.Parameters.AddWithValue("@Sostav", textBoxIng_dish.Text);

    try
    {
        command.ExecuteNonQuery();
    }
    catch (Exception err)

```

```

        {
            MessageBox.Show("Ошибка выполнения запроса.\n" + err.Message,
"Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        connection.Close();
        buttonSelectDishes_Click(this, EventArgs.Empty);
    }

```

Примечание:

Для того, чтобы загрузить фотографию из файла через `File.ReadAllBytes()` добавьте пространство имен:

```
using System.IO;
```

Покажем запрос на редактирование. Будем менять данные по блюду с указанным в соответствующей строке вводе коду блюда (`textBoxId_dish`).

Запрос будет иметь вид:

```

UPDATE Блюда
SET Название=@Name,
    Тип=@Type,
    Цена_меню=@Price,
    Выход=@Weight,
    Единицы=@Ed,
    Фирменное_блюдо=@Firm,
    Фото=@Photo,
    Состав=@Sostav
WHERE Код_блюда=@Id_dish

```

Следует отметить, что при редактировании данных пользователь для заданного кода блюда может изменять значения не во всех полях (в том числе и полях обязательных для заполнения при добавлении записи о новом блюде).

При логике редактирования, не указанные в строках ввода пользователем значения, остаются в БД не изменяемыми. Меняются только те, что указаны пользователем в строках ввода. В этом случае необходимо в зависимости от заполненных данных формировать и сам текст оператора UPDATE (формируемый запрос с параметрами).

Оформим данный запрос через соответствующий метод **UpdateDish()**:

Для СУБД Microsoft SQL Server:

```

void UpdateDish()
{
    if (String.IsNullOrEmpty(textBoxId_dish.Text))
    {
        MessageBox.Show("Обязательно укажите код блюда, для которого  
будете менять данные", "Внимание", MessageBoxButtons.OK,  
MessageBoxIcon.Warning);
        return;
    }

    int id;
    if (!int.TryParse(textBoxId_dish.Text, out id))
    {
        MessageBox.Show("Некоректное значение кода блюда!", "Внимание",  
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    double price=0;
    if ((!String.IsNullOrEmpty(textBoxPrice_dish.Text)) &&  
(!double.TryParse(textBoxPrice_dish.Text, out price)))
    {
        MessageBox.Show("Некоректное значение цены!", "Внимание",  
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    double weight=0;
    if ((!String.IsNullOrEmpty(textBoxWeight_dish.Text)) &&  
(!double.TryParse(textBoxWeight_dish.Text, out weight)))
    {
        MessageBox.Show("Некоректное значение веса!", "Внимание",  
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    string sqlUpdate = "UPDATE Блюда SET {0} [Фирменное_ блюдо]=@Firm  
WHERE Код_блюда=@Id_dish";
    SqlConnection connection = new  
SqlConnection(Properties.Settings.Default.CafeConnectionString);
    connection.Open();
    SqlCommand command = connection.CreateCommand();

    string sqlValues = "";

    if (!String.IsNullOrEmpty(textBoxName_dish.Text))
        sqlValues += "Название=@Name,";
    if (!String.IsNullOrEmpty(textBoxType_dish.Text))
        sqlValues += "Тип=@Type,";
    if (!String.IsNullOrEmpty(textBoxPrice_dish.Text))
        sqlValues += "Цена_меню=@Price,";
    if (!String.IsNullOrEmpty(textBoxWeight_dish.Text))
        sqlValues += "Выход=@Weight,";
    if (!String.IsNullOrEmpty(listBoxEd_dish.Text))
        sqlValues += "Единицы=@Ed,";
    if (!String.IsNullOrEmpty(fileImage))
        sqlValues += "Фото=@Photo,";
    if (!String.IsNullOrEmpty(textBoxIng_dish.Text))
        sqlValues += "Состав=@Sostav,";

    command.CommandText = String.Format(sqlUpdate, sqlValues);

    if (!String.IsNullOrEmpty(textBoxName_dish.Text))
        command.Parameters.AddWithValue("@Name", textBoxName_dish.Text);

```

```

//или другим способом, если необходимо явное указание типа данных
if (!String.IsNullOrEmpty(textBoxType_dish.Text))
    command.Parameters.Add("@Type", SqlDbType.NVarChar).Value =
textBoxType_dish.Text;

if (!String.IsNullOrEmpty(textBoxPrice_dish.Text))
    command.Parameters.AddWithValue("@Price", price);

if (!String.IsNullOrEmpty(textBoxWeight_dish.Text))
    command.Parameters.AddWithValue("@Weight", weight);

if (!String.IsNullOrEmpty(listBoxEd_dish.Text))
    command.Parameters.AddWithValue("@Ed", listBoxEd_dish.Text);

if (!String.IsNullOrEmpty(fileImage))
    command.Parameters.AddWithValue("@Photo",
File.ReadAllBytes(fileImage));

if (!String.IsNullOrEmpty(textBoxIng_dish.Text))
    command.Parameters.AddWithValue("@Sostav",
textBoxIng_dish.Text);

command.Parameters.AddWithValue("@Firm", checkBoxFirm.Checked);

command.Parameters.AddWithValue("@Id_dish", id);
try
{
    command.ExecuteNonQuery();
}
catch (Exception err)
{
    MessageBox.Show("Ошибка выполнения запроса:\n" + err.Message,
"Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
connection.Close();
buttonSelectDishes_Click(this, EventArgs.Empty);
}

```

Примечание:

В данном методе **UpdateDish()** формирование строки запроса на обновление идет в зависимости от заполненных пользователем данных в соответствующих компонентах, для которых потом подставляются соответствующие параметры (**формируемый параметрический запрос**).

Сбор строки запроса на обновление происходит при помощи метода **String.Format()**, который в указанную строку **sqlUpdate** подставляет указанную строку **sqlValues** в параметр, определенный в {0}, в данном случае в раздел после **SET**:

**UPDATE Блюда SET {0} [Фирменное_блюдо]=@Firm WHERE
Код_блюда=@Id_dish**

Следует отметить, что **Фирменное_блюдо** указывается в данном запросе всегда, так как определяется по состоянию **checkBoxFirm.Checked**, а

Код_блюда необходим для указания того, какое именно блюдо будет редактироваться.

Если убрать поле **Фирменное_блюдо** из запроса, то при формировании запроса строка **sqlValues** всегда заканчивается символом «,» что с полем **Фирменное_блюдо** является корректным, а без него приведет к ошибке синтаксиса оператора **UPDATE**.

Чтобы исключить данную последнюю запятую из текста запроса (в случае отсутствия поля **Фирменное_блюдо** в запросе) можно использовать:

```
sqlValues = sqlValues.Substring(0, sqlValues.Length - 1);
```

Удаление данных по блюду будем проводить по заданному пользователем коду блюда в строке ввода **textBoxId_dish**.

Запрос имеет вид:

DELETE FROM Блюда WHERE Код_блюда=@Id_dish

Оформим данный запрос через соответствующий метод **DeleteDish()**:

Для СУБД Microsoft SQL Server:

```
void DeleteDish()
{
    if (String.IsNullOrEmpty(textBoxId_dish.Text))
    {
        MessageBox.Show("Обязательно укажите код блюда данные которого  
необходимо удалить", "Внимание", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int id;
    if (!int.TryParse(textBoxId_dish.Text, out id))
    {
        MessageBox.Show("Некоректное значение кода блюда!", "Внимание",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

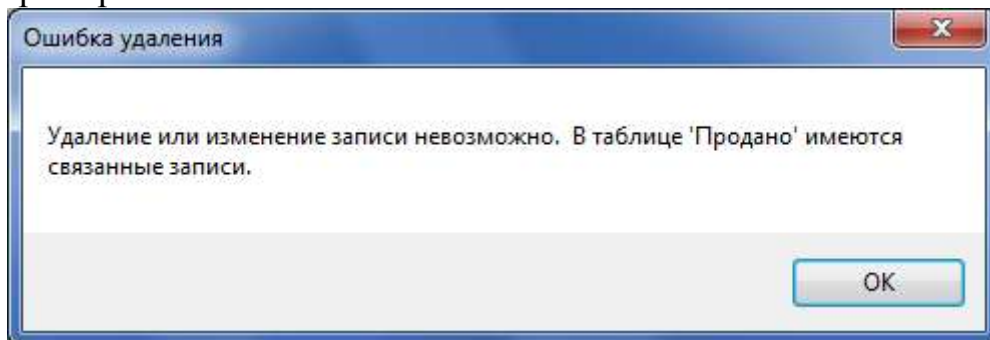
    string sqlDelete = @"DELETE FROM Блюда WHERE Код_блюда=@Id_dish";
    SqlConnection connection = new
    SqlConnection(Properties.Settings.Default.CafeConnectionString);
    connection.Open();
    SqlCommand command = connection.CreateCommand();
    command.CommandText = sqlDelete;
    command.Parameters.AddWithValue("@Id_dish", id);
    try
    {
        command.ExecuteNonQuery();
    }
    catch (Exception err)
    {
        MessageBox.Show(err.Message, "Ошибка удаления");
    }
}
```

```
}  
connection.Close();  
buttonSelectDishes_Click(this, EventArgs.Empty);  
}
```

Примечание:

Так как у нас в БД установлена ссылочная целостность без механизма каскадного удаления для **Блюда** и **Продано**, то возможно возникновение исключения при удалении блюда, указанного ранее в **Продано**. Для обработки этого исключения используем блок **try... catch**

Например:



Так как при удалении сотрудника нужна только строка для ввода Кода блюда, то пропишите для кнопки **Удалить данные по блюду с заданным кодом (radioButtonDelete_dish)**:

```
private void radioButtonDelete_dish_CheckedChanged(object sender, EventArgs e)  
{  
    panelDish.Visible = !radioButtonDelete_dish.Checked;  
}
```

Запрограммируем кнопку **Показать список блюд (buttonSelectDishes)**:

```
private void buttonSelectDishes_Click(object sender, EventArgs e)  
{  
    dataGridViewDish.DataSource = FillDataGridView("SELECT * FROM  
Блюда");  
    DataGridViewImageColumn column =  
(DataGridViewImageColumn)dataGridViewDish.Columns["Фото"];  
    column.ImageLayout = DataGridViewImageCellLayout.Stretch;  
}
```

Примечание:

в **dataGridViewDish** масштабируется фото.

Для кнопки **Выполнить запрос (buttonExecuteDML)** пропишите:

```
private void buttonExecuteDML_Click(object sender, EventArgs e)
```

```
{
    if (radioButtonInsert_dish.Checked)
        InsertDish();
    else
        if (radioButtonUpdate_dish.Checked)
            UpdateDish();
        else
            if (radioButtonDelete_dish.Checked)
                DeleteDish();
            else
                MessageBox.Show("Вы не выбрали действие", "Внимание",
    MessageBoxButtons.OK, MessageBoxIcon.Warning);
}
```

Запустите приложение. Протестируйте работу добавления, редактирования, удаления.

4 Задание на лабораторную работу №4

4.1 Изучить материал лабораторной работы №4.

4.2 Для своей БД необходимо выполнить:

4.2.1 использование запросов на выборку. Обязательно:

- выборка из одной таблицы;
- выборка с условиями;
- выборка из нескольких таблиц;
- агрегирующие (статистические) функции;
- полная запись оператора SELECT(предложения SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY);

- запросы с параметрами с проверкой корректности. Параметры разных типов данных;

4.2.2 использование подзапросов;

4.2.3 использование операторов изменения данных. Обязательно:

- INSERT;
- UPDATE;
- DELETE;

- параметрические запросы с проверкой корректности. Параметры разных типов данных;

4.2.4 протестировать работу полученного приложения.

5 Оформление отчета к лабораторной работе

Оформить отчёт со следующим содержанием:

5.1 Титульный лист.

5.2 Цель работы.

5.3 Постановка задачи.

5.4 Краткая теория и ход выполнения заданий.

5.5 Описание результатов.

Приведите скриншоты полученной программы и листинг программы.

5.6 Заключение (выводы).

Отчет должен быть оформлен согласно требованиям ОС ТУСУР.

6 Контрольные вопросы

6.1 Запись оператора SELECT.

6.2 Приведите примеры операторов, используемых в предложении WHERE.

6.3 Приведите примеры агрегирующих (статистических) функций, особенности их использования.

- 6.4** Как можно провести группировку данных в запросе?
- 6.5** Как можно провести упорядочивание/сортировку результатов запроса?
- 6.6** Особенности NULL значений в запросах.
- 6.7** Особенности перекрестного соединения таблиц.
- 6.8** Особенности внутреннего объединения таблиц.
- 6.9** Особенности левого объединения таблиц.
- 6.10** Особенности правого объединения таблиц.
- 6.11** Как провести объединение результатов выполнения нескольких операторов SELECT.
- 6.12** Запись оператора INSERT.
- 6.13** Запись оператора UPDATE.
- 6.14** Запись оператора DELETE.
- 6.15** Что такое подзапрос?
- 6.16** Особенности коррелированного и некоррелированного подзапроса.
- 6.17** Как можно задать текст SQL запроса в Microsoft Visual Studio?
- 6.18** Чем характерен динамический (параметрический) запрос?
- 6.19** Как можно задать нужные значения для параметров в Microsoft Visual Studio?